Graph Embeddings

Kaicheng YU

Westlake University, SoE

May 26, 2025

Table of Contents

1. Introduction to Graph Embeddings
2. Graph Neural Networks
3. Deep Graph Embeddings
4. Applications

Introduction to Graph Embeddings

- Focus: Data with semantic relationships between training samples $\{\mathbf{x}_n\}_{n=1}^{N}$.
- Relationships (edges) connect training samples (nodes) with application-specific meaning (commonly similarity).
- Graphs provide mathematical foundations for reasoning about these relationships.

Graphs as Universal Data Structures

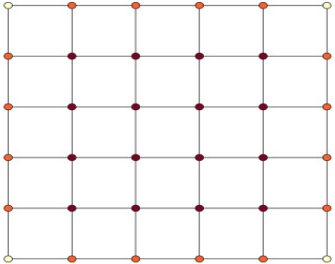
- Represent complex relational data (nodes and edges).
- Appear in multiple domains:
  - Social networks
  - Computational chemistry [Gil+17]
  - Biology [Sta+06]
  - Recommendation systems [KSJ09]
  - Semi-supervised learning [GB18]
- Notation:
  - $\mathbf{A} \in \{0,1\}^{N \times N}$: Adjacency matrix ($N$ = number of nodes).
  - $\mathbf{W} \in \mathbb{R}^{N \times N}$: Weighted version of adjacency matrix.
  - $\mathbf{X} \in \mathbb{R}^{N \times D}$: Matrix of node features.

Challenges in Graph Data

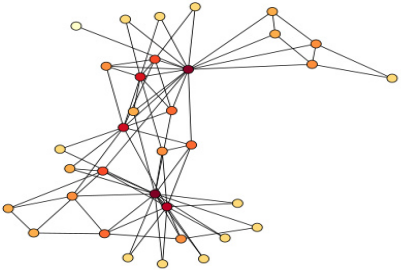
- Neural networks on graphs should apply to nodes in different graph settings (varying connections, community structure).
- Contrast with images: each pixel (node) has the same neighborhood structure.
- Arbitrary graphs: no specified alignment of nodes, each node might have different neighborhood structure.
- Euclidean spatial convolution cannot be directly applied on irregular graphs.
  - Euclidean convolutions rely on geometric priors (e.g., shift invariance) not generalizable to non-Euclidean domains.

Geometric Deep Learning (GDL)

- Developed to address challenges of applying deep learning to non-Euclidean data [Bro+17b].
- Graph Representation Learning (GRL) [Cha+21]:
  - Aims to learn low-dimensional continuous vector representations (embeddings) for graph-structured data.
- Figure 23.1: Euclidean vs. non-Euclidean graphs.



(a) Grid (Euclidean).



(b) Arbitrary graph (Non-Euclidean)

Classes of GRL Problems

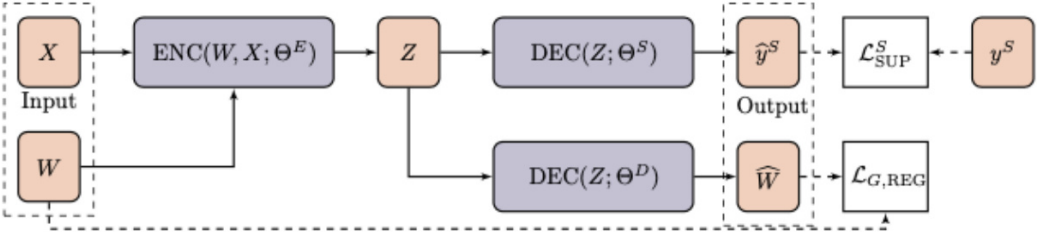
- **Unsupervised GRL**:
  - Learns low-dimensional Euclidean representations.
  - Objective: Preserve input graph structure.
- **Supervised (or Semi-supervised) GRL**:
  - Learns low-dimensional Euclidean representations.
  - Objective: For a specific downstream prediction task (node or graph classification).
- **Learning Settings**:
  - *Transductive*: Graph structure is fixed during training and testing (e.g., predicting user properties in a social network).
  - *Inductive*: Model answers questions about graphs not seen during training (e.g., classifying molecular structures).
- **Embedding Spaces**:
  - Most methods learn Euclidean vector spaces.
  - Recent interest in non-Euclidean representation learning (e.g., hyperbolic, spherical spaces).
  - Motivation: Use continuous embedding space resembling the discrete structure of input data (e.g., hyperbolic space for trees [Sar11]).

Graph Embedding as an Encoder/Decoder Problem

- Many GRL methods follow a similar pattern:
  1. Encode input (node features $\mathbf{X} \in \mathbb{R}^{N \times D}$ and graph edges $\mathbf{A}$ or $\mathbf{W} \in \mathbb{R}^{N \times N}$) from discrete domain to continuous representation (embedding) $\mathbf{Z} \in \mathbb{R}^{N \times L}$.
  2. Use learned representation $\mathbf{Z}$ to optimize an objective (e.g., reconstructing graph links).
- We use the Graph Encoder-Decoder Model (GraphEDM) by Chami et al. [Cha+21] to analyze GRL methods.

GraphEDM Framework

- Figure 23.2: Illustration of the GraphEDM framework [Cha+21].



- Encapsulates supervised and unsupervised methods, including:
  - Graph as regularizer [ZG02]
  - Positional embeddings [PARS14]
  - Graph neural networks (message passing [Gil+17; Sca+09], graph convolutions [Bru+14; KW16a])

GraphEDM Components: Input

- Input:
  - Weighted graph $\mathbf{W} \in \mathbb{R}^{N \times N}$
  - Optional node features $\mathbf{X} \in \mathbb{R}^{N \times D}$
- (Semi-)supervised settings: Training target labels for nodes (N), edges (E), or entire graph (G).
- Supervision signal $S \in \{N, E, G\}$.

GraphEDM Components: Networks

- **Graph encoder network** $\mathrm{ENC}_{\Theta^E} : \mathbb{R}^{N\times N} \times \mathbb{R}^{N\times D} \to \mathbb{R}^{N\times L}$:
  - Parameterized by Θ^E.
  - Combines graph structure and optional node features.
  - Produces node embedding matrix $\mathbf{Z} \in \mathbb{R}^{N\times L}$:

$$\mathbf{Z} = \mathrm{ENC}(\mathbf{W}, \mathbf{X}; \Theta^E)$$

  - Captures different graph properties depending on supervision.

GraphEDM Components: Networks (Cont.)

- **Graph decoder network** $\mathrm{DEC}_{\Theta^D} : \mathbb{R}^{N \times L} \to \mathbb{R}^{N \times N}$:
  - Parameterized by Θ^D.
  - Uses node embeddings $\mathbf{Z}$ to compute similarity scores for all node pairs in matrix $\widehat{\mathbf{W}} \in \mathbb{R}^{N \times N}$:

$$\widehat{\mathbf{W}} = \mathrm{DEC}(\mathbf{Z}; \boldsymbol{\Theta}^D)$$

- **Classification network** $\mathrm{DEC}_{\Theta^S} : \mathbb{R}^{N \times L} \to \mathbb{R}^{N \times |\mathcal{V}|}$ (where $\mathcal{V}$ is label space):
  - Used in (semi-)supervised settings, parameterized by Θ^S.
  - Output: distribution over labels $\hat{y}^S$, using node embeddings:

$$\widehat{\boldsymbol{y}}^S = \mathrm{DEC}(\mathbf{Z}; \boldsymbol{\Theta}^S)$$

- Specific choices of encoder/decoder networks realize specific graph embedding methods.

GraphEDM: Output and Loss Function

- Output:
  - Reconstructed graph similarity matrix $\widehat{W}$ (unsupervised).
  - Labels $\widehat{y}^S$ for supervised applications (node, edge, or graph level).
- Loss function optimizes parameters $\Theta = \{\Theta^E, \Theta^D, \Theta^S\}$:

$$\mathcal{L} = \alpha \mathcal{L}^S_{\text{SUP}}(y^S, \hat{y}^S; \mathbf{\Theta}) + \beta \mathcal{L}_{\text{G,RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \mathbf{\Theta}) + \gamma \mathcal{L}_{\text{REG}}(\mathbf{\Theta})$$

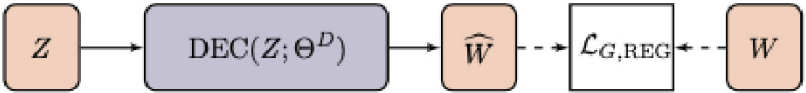
- α, β, γ are hyperparameters.
- Training can be supervised ($\alpha \neq 0$) or unsupervised ($\alpha = 0$).
  - Supervised: Leverages additional information (node/graph labels).
  - Unsupervised: Relies only on graph structure.

Shallow Graph Embeddings

- Transductive graph embedding methods.
- Encoder function maps categorical node IDs to a Euclidean space via an embedding matrix.
- Each node $v_i \in V$ has a low-dimensional learnable embedding vector $\mathbf{Z}_i \in \mathbb{R}^L$.
- Shallow encoder function:

$$\mathbf{Z} = \mathrm{ENC}(\Theta^E) \triangleq \Theta^E \quad \text{where} \quad \Theta^E \in \mathbb{R}^{N \times L}$$

- Embedding dictionary $\mathbf{Z}$ is directly learned as model parameters.
- **Unsupervised case**: Embeddings $\mathbf{Z}$ optimized to recover information about the input graph (e.g., adjacency matrix $\mathbf{W}$ or its transformation). Similar to dimensionality reduction (e.g., PCA) but for graph data.
- **Supervised case**: Embeddings optimized to predict labels (nodes, edges, or whole graph).
- Figure 23.3: Shallow embedding methods. Encoder is an embedding look-up; graph structure used only in the loss.



Unsupervised Shallow Embeddings: Types

- Two main types:

1. **Distance-based methods**:
   - Optimize embedding dictionary $\mathbf{Z} = \Theta^E \in \mathbb{R}^{N \times L}$.
   - Goal: Nodes i, j close in the graph (by graph distance) are embedded such that $d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ is small.
   - $d_2(\cdot, \cdot)$ is a pairwise distance function between embedding vectors (can be Euclidean or non-Euclidean).
   - Decoder output: $\widehat{\mathbf{W}} = \mathrm{DEC}(\mathbf{Z}; \mathbf{\Theta}^D)$, with $\widehat{W}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j)$.
2. **Outer product-based methods**:
   - Rely on pairwise dot products for node similarities.
   - Decoder network: $\widehat{\mathbf{W}} = \mathrm{DEC}(\mathbf{Z}; \Theta^D) = \mathbf{Z}\mathbf{Z}^\top$.

Reconstruction loss function:

$$\mathcal{L}_{\mathrm{G,RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \mathbf{\Theta}) = d_1(s(\mathbf{W}), \widehat{\mathbf{W}})$$

- $s(\mathbf{W})$: optional transformation of adjacency matrix $\mathbf{W}$.
- d_1: pairwise distance function between matrices (not necessarily same as d_2).
- Examples: $s(\mathbf{W}) = \mathbf{W}$ or $s(\mathbf{W}) = \mathbf{W}^2$. If $\mathbf{W} = \mathbf{A}$ (binary), use $s(\mathbf{W}) = 1 - \mathbf{W}$ to set connected nodes' distance to 0.

Shallow Embeddings: Distance-based Euclidean Methods

- Minimize Euclidean distances between similar (connected) nodes.
- **Multi-dimensional Scaling (MDS, Sec. 20.4.4)**:
  - $s(\mathbf{W})$: distance matrix (e.g., shortest path distances).
  - $d_1(s(\mathbf{W}), \widehat{\mathbf{W}}) = \sum_{i,j}(s(\mathbf{W})_{ij} - \widehat{\mathbf{W}}_{ij})^2 = \|s(\mathbf{W}) - \widehat{\mathbf{W}}\|_F^2$
  - $\widehat{\mathbf{W}}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j) = \|\mathbf{Z}_i - \mathbf{Z}_j\|$
- **Laplacian Eigenmaps (Sec. 20.4.9)**: Solves generalized eigenvector problem:

$$\min_{\mathbf{Z}\in\mathbb{R}^{|V|\times d}} \operatorname{tr}(\mathbf{Z}^\top \mathbf{L}\boldsymbol{\Lambda}\mathbf{Z}) \quad \text{s.t.} \quad \mathbf{Z}^\top\mathbf{D}\mathbf{Z} = \mathbf{I}, \quad \mathbf{Z}^\top\mathbf{D}\mathbf{1} = \mathbf{0}$$

- where $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is graph Laplacian, and $\mathbf{D}$ is the degree matrix.
- Equivalent: $\operatorname{tr}(\mathbf{Z}^\top\mathbf{L}\mathbf{Z}) = \frac{1}{2}\sum_{i,j} W_{ij}\|\mathbf{Z}_i - \mathbf{Z}_j\|_2^2$
- Reconstruction: $d_1(s(\mathbf{W}), \widehat{\mathbf{W}}) = \sum_{i,j}\mathbf{W}_{ij}\cdot\widehat{\mathbf{W}}_{ij}$, with $\widehat{\mathbf{W}}_{ij} = \|\mathbf{Z}_i - \mathbf{Z}_j\|_2^2$, $s(\mathbf{W}) = \mathbf{W}$

Shallow Embeddings: Distance-based Non-Euclidean Methods

- Hyperbolic geometry is effective for tree/hierarchical graphs.
- Nickel and Kiela [NK17]: Poincaré model of hyperbolic space.
- Change $d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ to Poincaré distance:

$$d_{\text{Poincare}}(\mathbf{Z}_i, \mathbf{Z}_j) = \operatorname{arcosh}\left(1 + 2 \cdot \frac{\|\mathbf{Z}_i - \mathbf{Z}_j\|_2^2}{\left(1 - \|\mathbf{Z}_i\|_2^2\right)\left(1 - \|\mathbf{Z}_j\|_2^2\right)}\right)$$

- Optimization: minimize connected pairs, maximize disconnected:

$$d_1(\mathbf{W}, \widehat{\mathbf{W}}) = \sum_{i,j} \mathbf{W}_{ij} \log\left(\frac{e^{-\widehat{\mathbf{W}}_{ij}}}{\sum_{k|\mathbf{W}_{ik}=0} e^{-\widehat{\mathbf{W}}_{ik}}}\right)$$

- (Denominator via negative sampling)
- Requires Riemannian optimization [Bon13]
- Other models: Lorentz [NK18], mixed-curvature [Gu+18], hyperbolic skip-gram [CCD17]

Shallow Embeddings: Outer Product-based Matrix Factorization

- Learn low-rank embedding of similarity matrix $s(\mathbf{W})$.
- Common choices: $s(\mathbf{W}) = \mathbf{W}$, Laplacian, Katz, Common Neighbors, Adamic/Adar.
- Decoder: $\widehat{\mathbf{W}} = \mathrm{DEC}(\mathbf{Z}; \mathbf{\Theta}^D) = \mathbf{Z}\mathbf{Z}^\mathsf{T}$.
- Loss: $\mathcal{L}_{G,\mathrm{RECON}} = ||s(\mathbf{W}) - \widehat{\mathbf{W}}||_F^2$.
- Graph Factorization (GF) [Ahm+13]: minimize $\sum_{(v_i,v_j)\in E}(\mathbf{W}_{ij} - \widehat{\mathbf{W}}_{ij})^2$.
- If $\mathbf{A}$ is binary: use sparse loss $||\mathbf{A} \circ (\mathbf{W} - \widehat{\mathbf{W}})||_F^2$.
- GraRep [CLX15]: directed graphs, learns source $\mathbf{Z}^s$ and target $\mathbf{Z}^t$ embeddings.
- k-hop: $\widehat{\mathbf{W}}^{(k)} = \mathbf{Z}^{(k),s}(\mathbf{Z}^{(k),t})^\mathsf{T}$.
- Loss per k: $||\mathbf{D}^{-k}\mathbf{W}^k - \widehat{\mathbf{W}}^{(k)}||_F^2$.
- Final embedding: concatenation across k.
- Not scalable (dense $\mathbf{D}^{-1}\mathbf{W}$), can use implicit matrix factorization [Per+17].

Shallow Embeddings: Outer Product-based Skip-gram Methods (1/2)

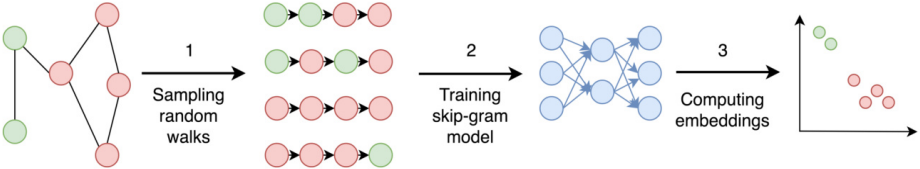
- Inspired by NLP (Word2Vec [Mik+13c; PSM14b]): predict context words for a target word.
- Objective: $\mathcal{L} = -\sum_{-K \leq i \leq K, i \neq 0} \log \mathbb{P}(w_{k-i} \mid w_k)$
- **DeepWalk [PARS14]**:
  - Uses random walks on graphs, mimicking word frequency in language.
  - Decoder: outer product (Eq. 23.13).
  - Generates node sequences via truncated unbiased random walks.
  - Maximizes log-likelihood of predicting context nodes from center node.
  - Uses two representations per node (center and context) [AEHPAR17].
- Transition matrix expectation: $s(\mathbf{W}) = \mathbb{E}_q \left[(\mathbf{D}^{-1}\mathbf{W})^q \right]$, $q \sim \mathrm{Categorical}([1, \ldots, T_{\mathsf{max}}])$

Shallow Embeddings: Outer Product-based Skip-gram Methods (2/2)

- Training minimizes:

$$\mathcal{L}_{G,\text{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \Theta) = \log Z(\mathbf{Z}) - \sum_{v_i, v_j} s(\mathbf{W})_{ij} \widehat{\mathbf{W}}_{ij}$$

$$\widehat{\mathbf{W}} = \mathbf{Z}\mathbf{Z}^{\mathsf{T}}, \quad Z(\mathbf{Z}) \text{ via hierarchical softmax or negative sampling}$$



- Figure 23.4: Pipeline for random-walk graph embedding methods.
- Skip-gram as implicit matrix factorization [LG14, Qiu+18] (NetMF framework: DeepWalk, LINE, node2vec).

Shallow Embeddings: Supervised Embeddings (1/2)

- Combines embedding learning with predicting node/graph labels.
- **Label Propagation (LP) [ZG02]**:
  - Semi-supervised node classification.
  - Encoder: shallow lookup table $\mathbf{Z}$.
  - Decoder: identity function $\hat{y}^N = \mathrm{DEC}(\mathbf{Z}; \Theta^C) = \mathbf{Z}$.
  - Smooths label distribution using graph structure (e.g., Laplacian eigenmaps).
  - Assumes neighbor nodes have similar labels; minimizes energy iteratively.
- Reconstruction loss: $\mathcal{L}_{G,\mathrm{RECON}}(\mathbf{W}, \widehat{\mathbf{W}}; \mathbf{\Theta}) = \sum_{i,j} \mathbf{W}_{ij} \|y_i^N - \hat{y}_j^N\|_2^2$

Shallow Embeddings: Supervised Embeddings (2/2)

- **Label Spreading (LS) [Zho+04]**: variant of LP with a different energy function.
- Reconstruction loss:

$$\mathcal{L}_{G,\text{RECON}} = \sum_{i,j} \mathbf{W}_{ij} \left\| \frac{\hat{y}_i^N}{\sqrt{D_i}} - \frac{\hat{y}_j^N}{\sqrt{D_j}} \right\|_2^2$$

- D_i: degree of node v_i.
- Supervised loss (for both LP and LS):

$$\mathcal{L}_{\text{SUP}}^N(y^N, \hat{y}^N; \Theta) = \sum_{i|v_i \in V_L} \|y_i^N - \hat{y}_i^N\|_2^2$$

Graph Neural Networks (GNNs)

- Define convolutions over graph data. [Cha+21]
- (Semi-)supervised neighborhood aggregation methods.
- Encoder: $\mathbf{Z} = \mathrm{ENC}(\mathbf{X}, \mathbf{W}; \Theta^E)$
- Decoders: $\widehat{\mathbf{W}} = \mathrm{DEC}(\mathbf{Z}; \mathbf{\Theta}^D)$ and/or $\widehat{\boldsymbol{y}}^S = \mathrm{DEC}(\mathbf{Z}; \mathbf{\Theta}^S)$

Message Passing GNNs

- Original GNN model [GMS05; Sca+09]: First deep learning for graphs.
- Views supervised graph embedding as information diffusion until equilibrium.
- Recursion with random initial embeddings $\mathbf{Z}^0$:

$$\mathbf{Z}^{t+1} = \mathrm{ENC}(\mathbf{X}, \mathbf{W}, \mathbf{Z}^t; \mathbf{\Theta}^E)$$

- Parameters Θ^E reused.
- After convergence ($t = T$), embeddings used for prediction:

$$\hat{y}^S = \mathrm{DEC}(\mathbf{X}, \mathbf{Z}^T; \mathbf{\Theta}^S)$$

- Parameters learned via backpropagation (Almeda-Pineda algorithm [Alm87; Pin88]).
- Converges if recursion is a contraction mapping (Banach's fixed point theorem).
- Scarselli et al. [Sca+09] use message passing:

$$\mathbf{Z}_i^{t+1} = \sum_{j|(v_i,v_j)\in E} f(\mathbf{X}_i, \mathbf{X}_j, \mathbf{Z}_j^t; \mathbf{\Theta}^E)$$

- $f(\cdot)$ is an MLP constrained to be a contraction mapping.

Message Passing GNNs (Cont.)

- **Gated Graph Sequence Neural Networks (GGSNNs) [Li+15]**:
  - Removes contraction mapping requirement from GNNs.
  - Relaxed recursion: Apply mapping functions (gated recurrent units [Cho+14b]) for fixed steps, parameters shared.
  - Outputs predictions at every step (useful for temporal graphs).
- **Message Passing Neural Networks (MPNNs) [Gil+17]**:
  - Framework for modern multi-layer GNNs with fixed layers.
  - Layer ℓ: Message functions $f^\ell(.)$ receive messages from neighbors, passed to aggregation $h^\ell(.)$:

$$\mathbf{m}_i^{\ell+1} = \sum_{j|(v_i,v_j)\in E} f^\ell(\mathbf{H}_i^\ell, \mathbf{H}_j^\ell)$$

$$\mathbf{H}_i^{\ell+1} = h^\ell(\mathbf{H}_i^\ell, \mathbf{m}_i^{\ell+1})$$

  - $\mathbf{H}^0 = \mathbf{X}$; after ℓ layers, hidden states encode ℓ-hop neighborhoods.
- **GraphNet [Bat+18]**:
  - Extends MPNNs to learn edge, node, and entire graph representations using message passing. Adds expressivity for wider application domains.

Spectral Graph Convolutions

- Define graph convolutions using the spectral domain of the graph Laplacian matrix.
- Two categories:
- **Spectrum-based methods** (e.g., Spectral CNNs [Bru+14]):
  - Explicitly compute eigendecomposition of the Laplacian.
  - Disadvantage: Domain-dependent (cannot generalize to new graphs), computationally expensive.
- **Spectrum-free methods** (e.g., GCN [KW16a]):
  - Motivated by spectral graph theory but no actual spectral decomposition.
  - Utilize approximations of spectral filters.
  - Disadvantage: Require using the whole graph $\mathbf{W}$, do not scale well.
- For more details, see [Bro+17b; Cha+21].

Spatial Graph Convolutions

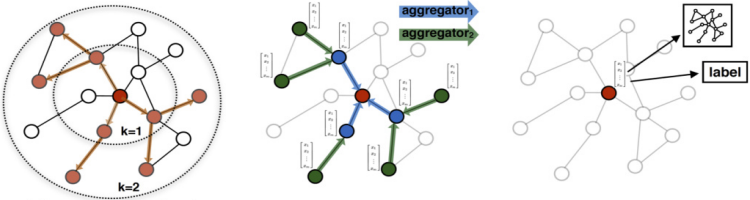
- Overcome limitations of spectral methods (domain dependency, scalability).
- Borrow ideas from standard CNNs: apply convolutions in the spatial domain defined by graph topology.
- Standard CNNs: Filters localized by fixed rectangular patches, pixel ordering allows weight reuse.
- Spatial GCNs: Use neighborhood sampling and attention to create fixed-size graph patches, overcoming irregularity.

Spatial Graph Convolutions: Sampling-based

- **GraphSAGE [HYL17]**: Inductive node embeddings.
  - Instead of averaging all 1-hop neighbors (Laplacian multiplication), SAGE samples fixed-size neighborhoods (q) for each node.
  - Removes dependency on fixed graph structure, allows generalization.
  - At each SAGE layer, nodes aggregate info from sampled neighbors (Figure 23.5).

$$\mathbf{H}_{:,i}^{\ell+1} = \sigma\left(\Theta_1^\ell \mathbf{H}_{:,i}^\ell + \Theta_2^\ell \mathrm{AGG}\left(\left\{\mathbf{H}_{:,j}^\ell \mid v_j \in \mathrm{Sample}(\mathrm{nbr}(v_i), q)\right\}\right)\right)$$

- AGG$(\cdot)$: Permutation invariant operator (e.g., SAGE-mean, SAGE-pool).
- Reduces computational complexity by working with fixed-size neighborhoods.
- Figure 23.5: GraphSAGE model.



1. Sample neighborhood
2. Aggregate feature information
3. Predict graph context and label

Spatial Graph Convolutions: Attention-based

- Attention mechanisms (Sec. 15.4) identify relevant parts of input (e.g., long sequences in language models).
- Applied to GCNs: Learn to focus on important neighbors during message passing via parametric patches learned on node features.
- More flexibility in inductive settings vs. fixed weights (GCNs).
- **Graph Attention Network (GAT) [Vel+18]**:
  - Attention-based GCNs.
  - Attends over neighborhood, selectively picks nodes for downstream task.
  - Similar intuition to SAGE [HYL17], suitable for inductive/transductive problems.
  - Unlike SAGE (fixed-size neighborhoods), GAT attends over all neighbors, assigning different weights.
  - Attention parameters trained via backpropagation, scores row-normalized with softmax.

Spatial Graph Convolutions: Geometric

- **MoNet [Mon+17]**: General framework, works well for features in geometric space (e.g., 3D point clouds, meshes).
- Learns attention patches using parametric functions in a pre-defined spatial domain (e.g., spatial coordinates).
- Applies convolution filters in the resulting graph domain.
- Generalizes methods like Geodesic CNN (GCNN) [Mas+15] and Anisotropic CNN (ACNN) [Bos+16].
- MoNet is more general: any pseudo-coordinates (node features) can induce patches.
- Layer:

$$\mathbf{H}^{\ell+1} = \sigma\left(\sum_{k=1}^{K}(\mathbf{W} \odot g_k(\mathbf{U}^s))\mathbf{H}^\ell \Theta_k^\ell\right)$$

- Gaussian kernel for patches:

$$g_k(\mathbf{U}^s) = \exp\left(-\frac{1}{2}(\mathbf{U}^s - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{U}^s - \boldsymbol{\mu}_k)\right)$$

with $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$ learned, $\boldsymbol{\Sigma}_k$ diagonal.

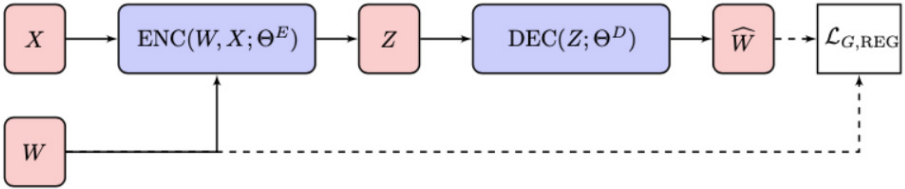
Non-Euclidean Graph Convolutions

- Extends GNNs to learn non-Euclidean embeddings (e.g., hyperbolic for hierarchical graphs [Sec 23.3.3]).
- Challenge: No standard inner product/matrix multiplication in non-Euclidean space.
- **HGCN [Cha+19a]** and **HGNN [LNK19]**:
  - Perform graph convolutions in tangent space (Euclidean approximation at a point).
  - Process: map to tangent space $\rightarrow$ apply convolution $\rightarrow$ map back to hyperbolic.
  - Works well on hierarchical graphs (see Figure 23.6).
- Figure 23.6: Euclidean vs. hyperbolic embeddings of a tree graph.

(a) GCN layers. *(b) HGCN layers.*

Deep Graph Embeddings: Unsupervised

- Using GNNs for unsupervised and semi-supervised graph embeddings.
- Figure 23.7: Structure + features → GNN encoder → embeddings → graph regularization loss.



Unsupervised Deep Embeddings: SDNE

- **SDNE [WCZ16]**: Auto-encoder preserving 1st 2nd order proximity.
- Input: row of adjacency matrix ($s(\mathbf{W}) = \mathbf{W}$), ignores features.
- Output: $\mathbf{Z} = \mathrm{ENC}(\mathbf{W}; \Theta^E)$, then $\widehat{\mathbf{W}} = \mathrm{DEC}(\mathbf{Z}; \Theta^D)$.
- Loss:

$$||(s(\mathbf{W}) - \widehat{\mathbf{W}}) \cdot \mathbb{I}(s(\mathbf{W}) > 0)||_F^2 + \alpha_{\mathrm{SDNE}} \sum_{ij} s(\mathbf{W})_{ij} ||\mathbf{Z}_i - \mathbf{Z}_j||_2^2$$

Unsupervised Deep Embeddings: (V)GAE

- **GAE / VGAE [KW16b]**: Use graph convolutions for encoding.
- $\mathbf{Z} = \mathrm{GCN}(\mathbf{W}, \mathbf{X}; \Theta^E)$, decoder: $\widehat{\mathbf{W}} = \mathbf{Z}\mathbf{Z}^\mathsf{T}$
- Reconstruction loss:

$$\mathcal{L}_{G,\mathrm{RECON}} = -\sum_{i,j} (1 - \mathbf{W}_{ij}) \log(1 - \sigma(\widehat{\mathbf{W}}_{ij})) + \mathbf{W}_{ij} \log \sigma(\widehat{\mathbf{W}}_{ij})$$

- **VGAE**: Uses variational inference with prior $p(\mathbf{Z}) = \mathcal{N}(0, I)$.
- Inference model $q_\Phi(\mathbf{Z}|\mathbf{W}, \mathbf{X})$, minimizes:

$$\mathrm{NELBO} = -\mathbb{E}_q[\log p(\mathbf{W}|\mathbf{Z})] + \mathrm{KL}(q||p)$$

Unsupervised Deep Embeddings: Graphite & Contrastive

- **Graphite [GZE19]**: Iterative decoding + GCN:

$$\widehat{\mathbf{W}}^{(k)} = \frac{\mathbf{Z}^{(k)}\mathbf{Z}^{(k)\mathsf{T}}}{||\mathbf{Z}^{(k)}||_2^2} + \frac{\mathbf{1}\mathbf{1}^{\mathsf{T}}}{N} \quad \mathbf{Z}^{(k+1)} = \mathrm{GCN}(\widehat{\mathbf{W}}^{(k)}, \mathbf{Z}^{(k)})$$

- **DGI [Vel+19]**: Discriminator maximizes mutual info:

$$\min_{\Theta} -\mathbb{E}\sum_i \log \mathcal{D}(\mathbf{Z}_i, \mathcal{R}(\mathbf{Z})) - \sum_j \log(1 - \mathcal{D}(\mathbf{Z}_j^-, \mathcal{R}(\mathbf{Z}^-)))$$

- **GMI [Pen+20]**: Maximizes MI between node and neighborhood embeddings.

Deep Graph Embeddings: Semi-supervised

- **SemiEmb [WRC08]**:
- $\mathbf{Z} = \mathrm{ENC}(\mathbf{X}; \Theta^E)$ via MLP.
- Decoder: $\widehat{\mathbf{W}}_{ij} = ||\mathbf{Z}_i - \mathbf{Z}_j||^2$
- Regularize hidden layers via label propagation loss.
- **Planetoid [YCS16]**: Combines random walks + supervised info.
- $\mathbf{Z} = [\mathbf{Z}^c || \mathbf{Z}^F]$, learn via:

$$\mathcal{L}_{\mathrm{G,RECON}} = -\mathbb{E}_{(i,j,\gamma)} \log \sigma(\gamma \widehat{\mathbf{W}}_{ij}), \quad \widehat{\mathbf{W}}_{ij} = \mathbf{Z}_i \mathbf{Z}_j^{\mathsf{T}}$$

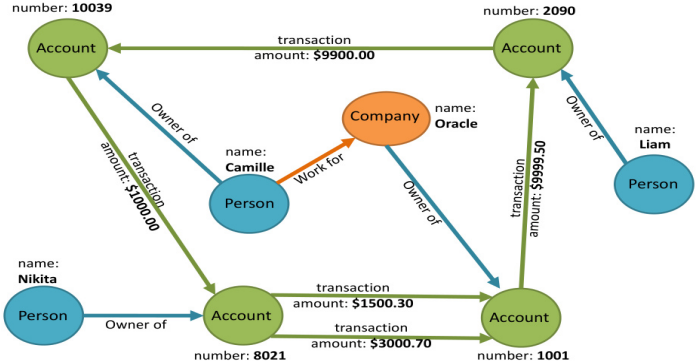
$$\mathcal{L}^N_{\mathrm{SUP}} = -\frac{1}{|V_L|} \sum_{i \in V_L} \sum_k y^N_{ik} \log \widehat{y}^N_{ik}$$

Unsupervised Applications

- **Graph Reconstruction**: Embeddings to reconstruct original graph structure.
- **Link Prediction**: Predict missing or future links.
- Use case: Fraud detection, recommendation, knowledge graphs [Nic+15].

Unsupervised Applications (Cont.)

- Figure 23.8: Graph of financial transactions.



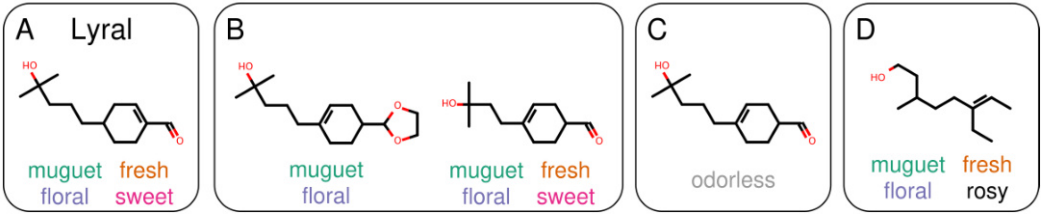
- **Clustering**: Community detection via k-means on embeddings.
- **Visualization**: t-SNE/PCA of embeddings for interpretation.

Supervised Applications: Node Classification

- Predict node labels using learned embeddings.
- **Semi-supervised setting**: Partially labeled graph.
- **Inductive setting**: New nodes/graphs at test time.
- Strong performance from combining structure + features (e.g., GCN, GraphSAGE).

Supervised Applications: Graph Classification

- Predict graph-level labels (e.g., molecule properties).
- Pooling needed to get graph-level features.
- Figure 23.9: Molecules and odor descriptors.



- Can also use unsupervised methods [Tsi+18; ARZP19; TMP20].